

```

// פונקציה עזר
// מקבלת רשימה וערך
// מחזירה אמת עם הערך נימצא ברשימה
1 public static boolean contains(Node<Integer> node, int x) {
2     while (node != null) {
3         if (node.getValue() == x) {
4             return true;
5         }
6         node = node.getNext();
7     }
8     return false;
9 }

```

```

1 public static Node<Integer> copyEvens(Node<Integer> node) {
2     Node<Integer> lst = null;
3     while (node != null) {
4         if (node.getValue() % 2 == 0) {
5             lst = new Node<Integer>(node.getValue(), lst);
6         }
7         node = node.getNext();
8     }
9     return lst;
10 }

```

```

1 public static int countBoth(Node<Integer> a, Node<Integer>
2     int count = 0;
3     while (a != null) {
4         count++;
5         a = a.getNext();
6     }
7     while (b != null) {
8         count++;
9         b = b.getNext();
10    }
11    return count;
12 }

```

כתבו פעולה חיצונית בשם createUnion בשפת Java או CreateUnion בשפת C#. הפעולה מקבלת כקלט את שתי הרשימות ומחזירה רשימה חדשה המכילה את כל המספרים המופיעים לפחות באחת מהרשימות.

סדר המספרים ברשימה המוחזרת אינו חשוב וכל מספר מופיע פעם אחת בלבד.

```

1 public static Node<Integer> createUnion(Node<Integer> a,
2     Node<Integer> b)
3 {
4     Node<Integer> lst = null;
5     while (a != null) {
6         if (!contains(lst, a.getValue())) {
7             lst = new Node<Integer>(a.getValue(), lst);
8         }
9         a = a.getNext();
10    }
11    while (b != null) {
12        if (!contains(lst, b.getValue())) {
13            lst = new Node<Integer>(b.getValue(), lst);
14        }
15        b = b.getNext();
16    }
17    return lst;
18 }

```

כתבו פעולה חיצונית בשם intersection בשפת Java או בשפת C#.
 הפעולה מקבלת כקלט את שתי הרשימות ומחזירה רשימה חדשה המכילה רק את המספרים
 המופיעים גם ב-lst1 וגם ב-lst2.
 סדר המספרים ברשימה המוחזרת אינו חשוב וכל מספר מופיע פעם אחת בלבד. אם אין אף מספר
 משותף, תוחזר רשימה ריקה.

```
1 public static Node<Integer> intersection(Node<Integer> a,
2                                         Node<Integer> b)
3 {
4     Node<Integer> lst = null;
5     while (a != null) {
6         int x = a.getValue();
7         if (contains(b, x) && !contains(lst, x)) {
8             lst = new Node<Integer>(x, lst);
9         }
10        a = a.getNext();
11    }
12    return lst;
13 }
```

שני תורים הם "M-מחברים" אם ערכם של M האיברים האחרונים של התור הראשון זהים (לפי אותו
 סדר) לערכם של-M האיברים הראשונים של התור השני.

```
1 // The last M numbers of a[] are first M numbers of b[]
2 public static boolean isMConnect(int[] a, int[] b, int m) {
3     int k = a.length - m;
4     for (int i = 0; i < m; i++) {
5         if (a[i + k] != b[i]) {
6             return false;
7         }
8     }
9     return true;
10 }
11
12 int[] a = new int[] {2,5,4,4,7,2,6};
13 int[] b = new int[] {2,6,7,1,4,2,1,8,1};
14 print(isMConnect(a, b, 2));
15
16 // Runtime complexity of isMConnect() is O(n)
```

פונקציה עזר... מה מקבלת ומה מחזירה...
 1 public static int atIndex(Queue<Integer> que, int indx) {
 2 Queue<Integer> tmp = new Queue<Integer>();
 3 while (indx > 0) {
 4 tmp.insert(que.remove());
 5 indx--;
 6 }
 7 int x = que.remove();
 8 tmp.insert(x);
 9 while (!que.isEmpty()) {
 10 tmp.insert(que.remove());
 11 }
 12 while (!tmp.isEmpty()) {
 13 que.insert(tmp.remove());
 14 }
 15 return x;
 16 }
 17 // Runtime complexity of atIndex() is O(n)

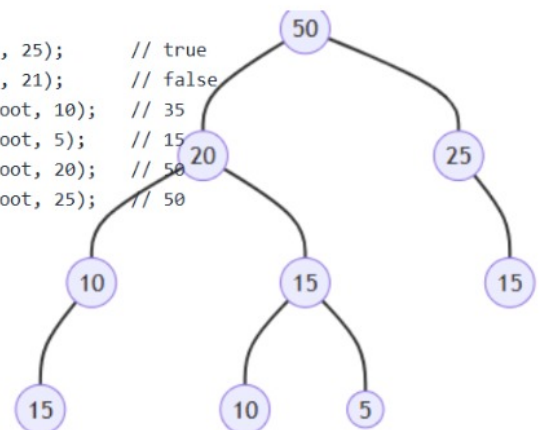
```
1 public static int maxConnect(int[] a, int[] b) {
2     int max = 0;
3     for (int i = 0; i < a.length; i++) {
4         if (isMConnect(a, b, i)) {
5             max = i;
6         }
7     }
8     return max;
9 }
10
11 int[] a = new int[] {2,5,4,4,7,2,6};
12 int[] b = new int[] {2,6,7,1,4,2,1,8,1};
13
14 print(maxConnect(a, b));           // 2
15 print(maxConnect(b, a));           // 0
16
17 // Runtime complexity of maxConnect is O(n^2)
```

```

1 public static boolean isMConnect(Queue<Integer> a,
2                                 Queue<Integer> b,
3                                 int m)
4 {
5     int k = length(a) - m;
6     for (int i = 0; i < m; i++) {
7         if (atIndex(a, i + k) != atIndex(b, i)) {
8             return false;
9         }
10    }
11    return true;
12 }
13
14 public static int maxConnect(Queue<Integer> a,
15                              Queue<Integer> b)
16 {
17     int max = 0;
18     for (int i = 0; i < length(a); i++) {
19         if (isMConnect(a, b, i)) {
20             max = i;
21         }
22     }
23     return max;
24 }
25
26 var a = queueFromArray(new Integer[] {2,5,4,4,7,2,6});
27 var b = queueFromArray(new Integer[] {2,6,7,1,4,2,1,8,1});
28
29 print(isMConnect(a, b, 2));    // true
30 print(maxConnect(a, b));      // 2
31 print(maxConnect(b, a));      // 0

```

```
1 sod(root, 25);    // true
2 sod(root, 21);    // false
3 secret(root, 10); // 35
4 secret(root, 5);  // 15
5 secret(root, 20); // 50
6 secret(root, 25); // 50
```



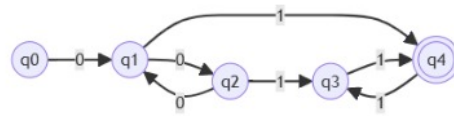
לפניכם השפות L_1 ו- L_2 מעל הא"ב $\{0,1\}$:

$$L_1 = \{0^k 1^n 0^m \mid k, n, m > 0, n + m \geq k\} \quad \Sigma: 010 \ 101 \ 0110 \ 00010 \ 001100$$

$$L_2 = \{0^k 1^n \mid n, k > 0, n \bmod 2 = k \bmod 2\} \quad \Sigma: 01 \ 000111 \ 1100 \ 00011 \ 0011$$

- א. כתבו מילה באורך 6 השייכת לשפה L_1 ומילה באורך 6 השייכת לשפה L_2 .
 ב. אם השפה L_1 רגולרית, יש לבנות אוטומט סופי דטרמיניסטי לא מלא שיקבל את השפה. אם השפה אינה רגולרית, יש לבנות אוטומט מחסנית דטרמיניסטי שיקבל את השפה.
 ג. אם השפה L_2 רגולרית, יש לבנות אוטומט סופי דטרמיניסטי לא מלא שיקבל את השפה. אם השפה אינה רגולרית, יש לבנות אוטומט מחסנית דטרמיניסטי שיקבל את השפה.

Q-7-L2



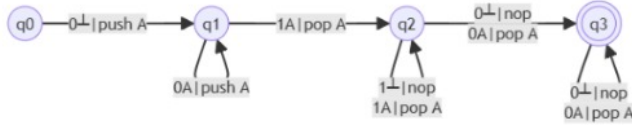
לפניכם השפות L_1 ו- L_2 מעל הא"ב $\{0,1\}$:

$$L_1 = \{0^k 1^n 0^m \mid k, n, m > 0, n + m \geq k\} \quad \Sigma: 010 \ 101 \ 0110 \ 00010 \ 001100$$

$$L_2 = \{0^k 1^n \mid n, k > 0, n \bmod 2 = k \bmod 2\} \quad \Sigma: 01 \ 000111 \ 1100 \ 00011 \ 0011$$

- א. כתבו מילה באורך 6 השייכת לשפה L_1 ומילה באורך 6 השייכת לשפה L_2 .
 ב. אם השפה L_1 רגולרית, יש לבנות אוטומט סופי דטרמיניסטי לא מלא שיקבל את השפה. אם השפה אינה רגולרית, יש לבנות אוטומט מחסנית דטרמיניסטי שיקבל את השפה.
 ג. אם השפה L_2 רגולרית, יש לבנות אוטומט סופי דטרמיניסטי לא מלא שיקבל את השפה. אם השפה אינה רגולרית, יש לבנות אוטומט מחסנית דטרמיניסטי שיקבל את השפה.

Q-7-L1



נתונה השפה L מעל הא"ב $\{0,1\}$ בה המילים מקיימות את שני התנאים הבאים:

דוגמאות למילים בשפה:

00, 01010, 101000100

דוגמאות למילים שאינן בשפה:

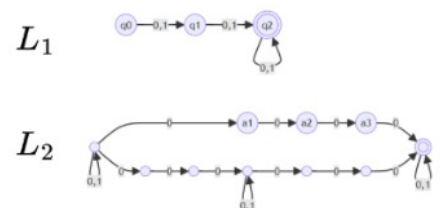
0000, 1, 1000110001

- אורך כל המילים בשפה הוא לפחות 2.
- הרצף 000 מופיע לכל היותר פעם אחת בלבד במילה.

הוכיחו שהשפה L רגולרית. מומלץ להשתמש בתכונות הסגורות.

אין כניסה לתלמידים שעשו לכל היותר תרגיל אחד בלבד!
 מי כן יכול להיכנס?

מי שעשה לפחות שני תרגילים



$$L = L_1 \cap \overline{L_2}$$

נתונות השפות L_1 , L_2 , L_3 מעל הא"ב $\{a,b\}$:

$$L_1 = \{ \text{כל המילים בהן מספר המופעים של האות } a \text{ והאות } b \text{ שווה} \}$$

$$L_2 = \{ \text{כל המילים בהן מספר המופעים של האות } a \text{ מופיע לפחות שתי פעמים} \}$$

$$L_3 = L_1 \cap L_2$$

aaabbb

(1) כתבו מילה באורך 6 או יותר השייכת לשפה L_1 .

aaabbb

(2) כתבו מילה באורך 6 או יותר השייכת לשפה L_2 .

(3) תגדירו במילים את השפה L_3 .

(4) מהו אורך המילה המינימלית בכל אחת מן השפות?

$$L_1: |a| = |b| = 0$$

$$L_2: |aa| = 2$$

$$L_3: |aabb| = 4$$

כל המילים בהן מספר המופעים של האות **a** והאות **b** שווה
 פחות מילה ריקה ומילים **ab** ו-**ba**

```

1 class OrderedList {
2     private Node<NumCount> lst;
3
4     public int valueN(int num) {
5         Node node = this.lst;
6         while (node != null) {
7             int count = node.getValue().getCount();
8             if (num < count) {
9                 return node.getValue().getNum();
10            }
11            num -= count;
12            node = node.getNext();
13        }
14        return node.getValue().getNum();
15    }
16 }

```

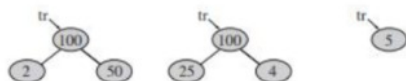
```

1 public static boolean isPrime(int num) {
2     for (int i = 2; i < num; i++) {
3         if (num % i == 0) {
4             return false;
5         }
6     }
7     return true;
8 }
9
10 public static factorOf(int num) {
11     for (int i = 2; i < num; i++) {
12         if (num % i == 0) {
13             return i;
14         }
15     }
16     return num;
17 }

```

הפעולה מקבלת צומת ללא בנים (עלה) שערכו גדול מ-0. אם ערך הצומת הוא מספר ראשוני, הפעולה מחזירה false. אחרת, הפעולה מוסיפה לצומת שני בנים שיערך המכפלה שלהם שווה לערך הצומת, והערך של כל אחד מהם גדול מ-1. לאחר ההוספה הפעולה מחזירה true.

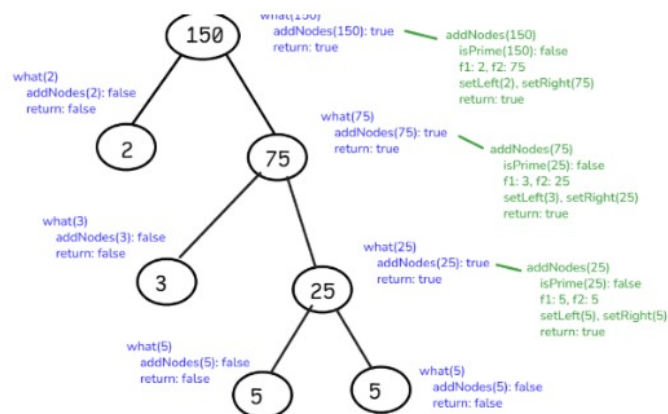
דוגמאות: בתום הפעולה הצמתים יכולים להיראות כך (עבור הערכים 100, 100, 5):



```

1 public static boolean addNodes(BinNode tr) {
2     int num = tr.getValue();
3     if (isPrime(num)) {
4         return false;
5     }
6     int f1 = factorOf(num);
7     int f2 = num / f1;
8     tr.setLeft(new BinNode(f1));
9     tr.setRight(new BinNode(f2));
10    return true;
11 }

```



```

public static void what (BinNode<Integer> tr)
{
    if (addNodes (tr))
    {
        what (tr.getLeft());
        what (tr.getRight());
    }
}

```